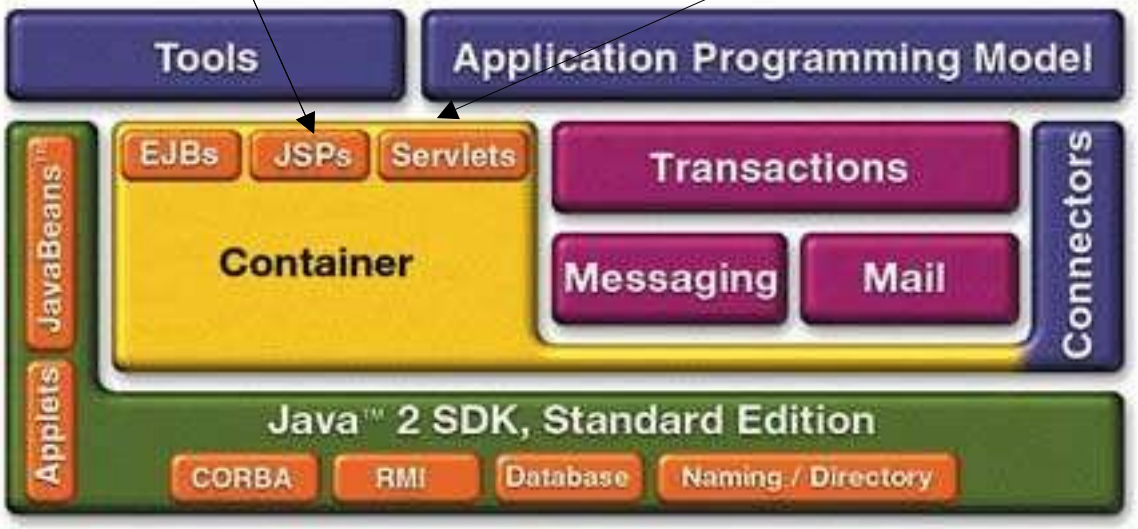
Java Server Pages

Server Side Technology

JSP and Servlet in J2EE Architecture

An extensible Web technology that uses template data, custom elements, scripting languages, and server-side Java objects to **return dynamic content to a client**. Typically the template data is HTML or XML elements. The client is often a **Web browser**.

Java Servlet A Java program that extends the functionality of a Web server, generating dynamic content and interacting with Web clients using a **request-response paradigm**.



Java Server Pages (JSP)

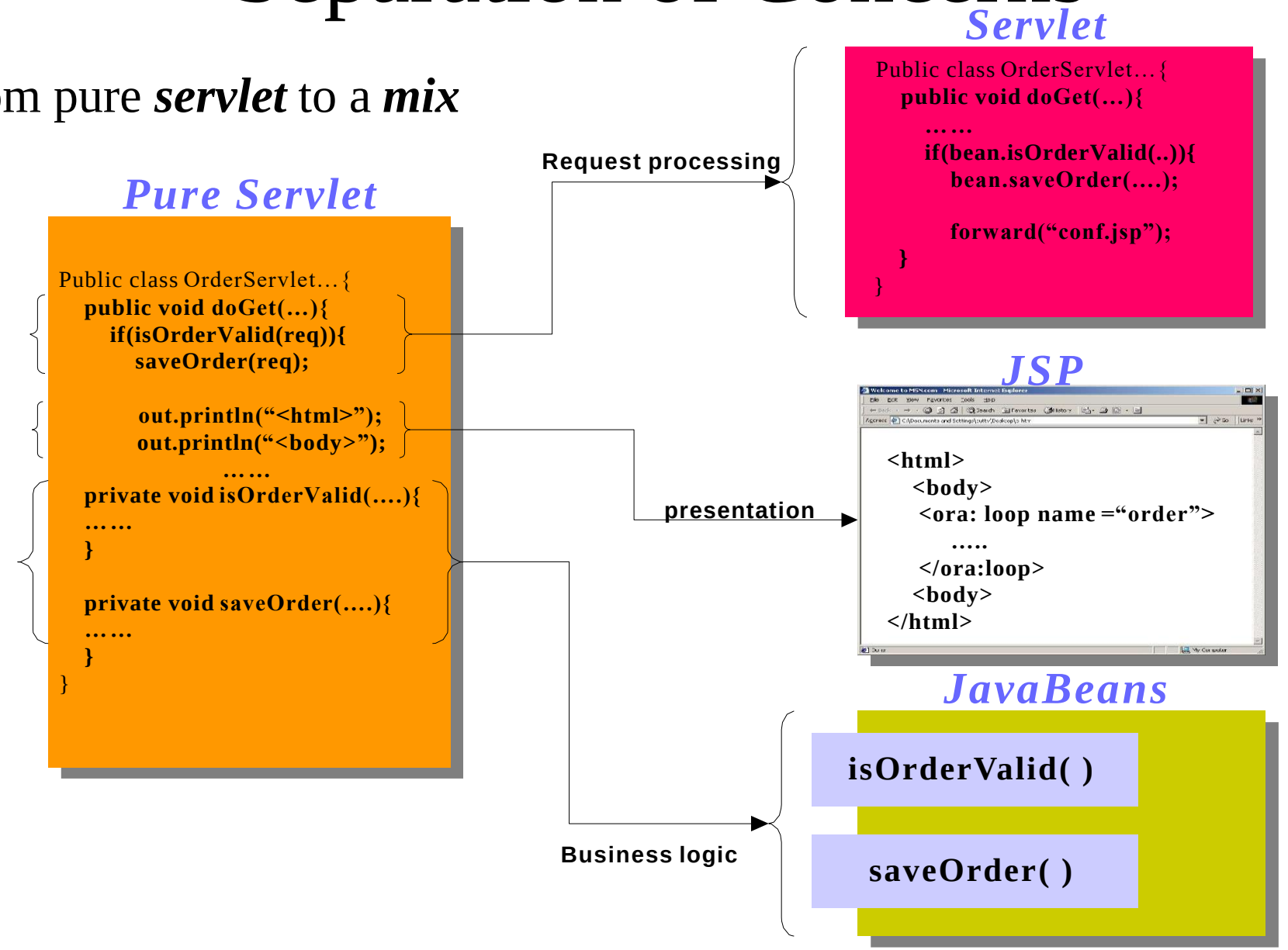
- JSP is a server side technology like a Servlet.
- Fast way to create web pages to display dynamic data.
- JSP page contain both static and dynamic data (JSP = HTML + Java)
- Help to write web application easily even with a less knowledge of java
- Initially JSP was developed to replace servlet but now common practice is to use servlet and JSP together using MVC (Model-View-Controller) pattern.

JSP versus Servlet

- Servlets
  - HTML Code in Java
  - Not easy to author (lot of println)
- JSP
  - Java like code in HTML
  - Easier to author
  - Compiled into servlet
- Both have pro and cons
  - typical use is to combine them (e.g. MVC pattern)

Separation of Concerns

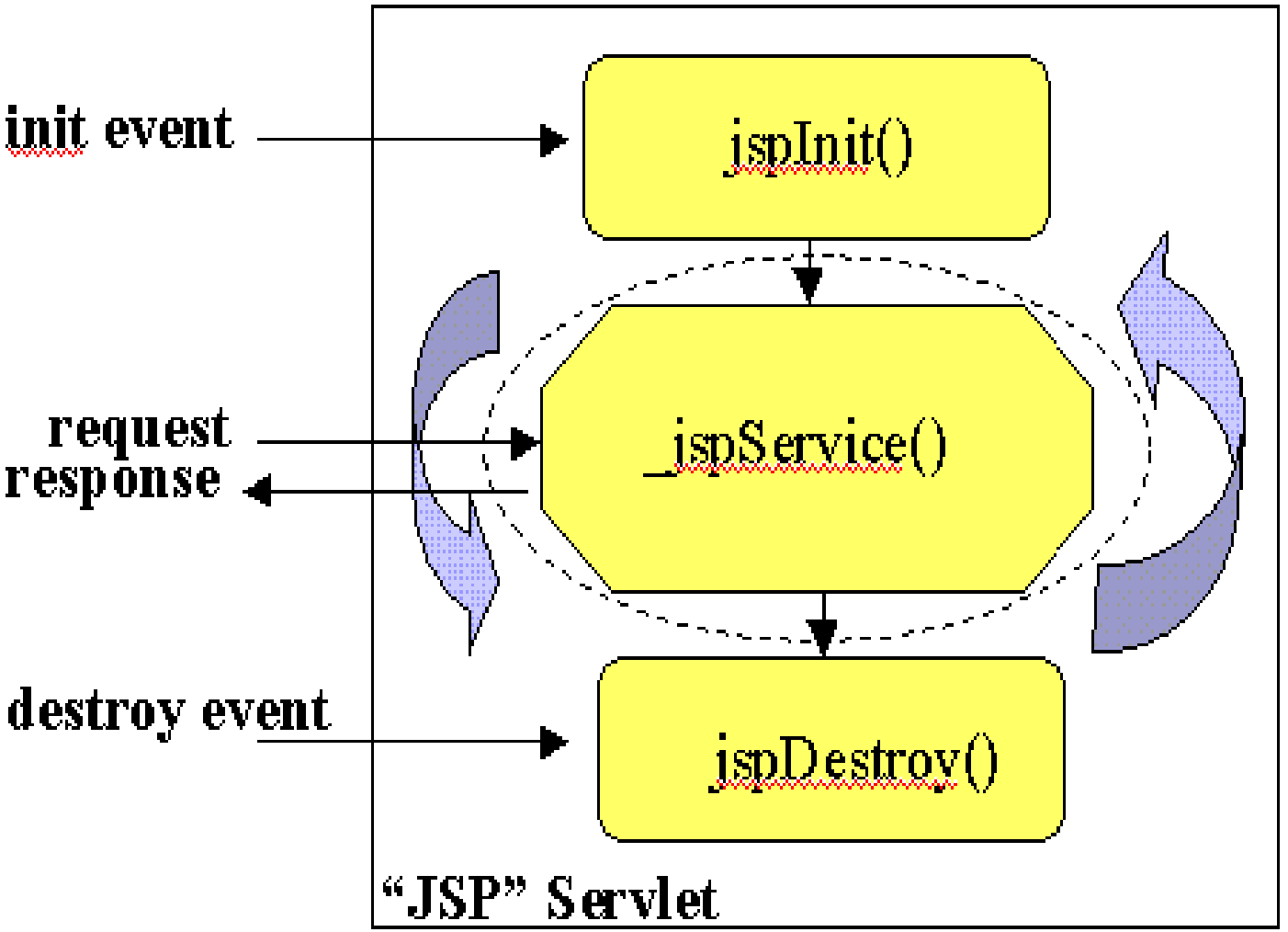
From pure ***servlet*** to a ***mix***



Lifecycle of a JSP

- A JSP life cycle can be defined as the entire process from its creation till the destruction.
- The following are the paths followed by a JSP
  - Compilation has three steps
    - Parsing JSP
    - Turning the JSP into a Servlet
    - Compiling the servlet
  - Initialization
  - Execution
  - Destroy

JSP Lifecycle



JSP Initialization

- Declare methods for performing the following tasks using JSP declaration mechanism
  - Read persistent configuration data
  - Initialize resources
  - Perform any other onetime activities
- By overriding ***jspInit()*** method of JspPage interface

JSP Finalization

- Declare methods for performing the following tasks using JSP declaration mechanism
  - Read persistent configuration data
  - Release resources
  - Perform any other onetime cleanup activities
- By overriding ***jspDestroy()*** method of JspPage interface

Structure of JSP Page

- The Structure of JSP page is a cross between a Servlet and Static web page(HTML) with java code enclosed between the constructs <% ……. %> and other XML like tags.
- There are three categories of JSP Tags
  - Directives
  - Scripting Elements
  - Actions

JSP Directives

- Messages to the JSP container in order to affect overall structure of the servlet
- JSP directives used to set global values such as class declaration, methods to be implemented, output content type and so on.
- Do not produce any output to the client.

<%@ directive-name attribute =“value” ……. %>

- There are three main directives
  - page
  - include
  - taglib

JSP Directives (Cont..)

Page Directive

- To define and manipulate a number of important attribute that affect the whole JSP page

ex. <%@ page language="java" import ="java.sql.*", session ="true"
autoflush="true" ……%>

Include Directive

- The include directives instructs the container to include the content of resource in the current JSP

<%include file="file name" %>

taglib directives

- Allows the page to use custom tags. It names the tag libraray that contain compiled java code defining tags to used

<% @ taglib uri="……" prefix ="……." %>

JSP Scripting Elements

- It allows to insert Java code into the servlet that will be generated from JSP page.
- i.e., It allows java code to be inserted into a jsp page like variables and method declarations.
- There are three forms
  - ✓ Expressions: <%= Expressions %>
  - ✓ Scriptlets: <% Code %>
  - ✓ Declarations: <%! Declarations %>

JSP Scripting Elements : Expressions

- Java Code placed with expression tag
- Expression is evaluated and converted into a String
- The String is then Inserted into the servlet's output stream of the response. Results in something like out.println(expression)
- Syntax

  <%=statement %?
- Example

```
<html > <body>
  <%="Hello World!" %>
</body> </html>
```

JSP Scripting Elements : Scriptlet tag

- A JSP, Java Code can be written inside the JSP page using Scriptlet tag
- Syntax :
  - <% Java Source code %>
- Example

```
<html>
    <body>
        <%  out.print("Hello world"); %>
    </body>
</html>
```

JSP Scripting Elements : Declarations

- To define variables or methods that get inserted into the main body of servlet class
- For initialization and cleanup in JSP pages, used to override jspInit() and jspDestroy() methods
- Format:

  <%! method or variable declaration code %>

  <jsp:declaration> method or variable declaration code

  </jsp:declaration>

Including Content in JSP Page

- Include directive
  - content that need to be reused in other pages (e.g. banner content, copyright information)

    <%@ include file="banner.jsp" %>
- jsp:include element
  - to include either a static or dynamic resource in a JSP file
    - static: its content is inserted into the calling JSP file
    - dynamic: request is sent to the included resource, the included page is executed, and then the result is included in the response from the calling JSP page
  - <jsp:include page="date.jsp"/>

Forwarding

- Same mechanism as for servlets
- Syntax

```
<jsp:forward page="/main.jsp" />
```

- Original request object is provided to the target page via jsp:parameter element

```
<jsp:include page="..." >

<jsp:param name="param1" value="value1"/>

</jsp:include>
```

Directives

- Messages to the JSP container in order to affect overall structure of the servlet
- Do not produce output into the current output stream
- Syntax

<%@ directive {attr=value}* %>

Implicit Objects

- JSP page has access to a number of implicit objects (same as servlets)
  - request (HttpServletRequest)
  - response (HttpServletRepsonse)
  - session (HttpSession)
  - application(ServletContext)
  - out (of type JspWriter)
  - config (ServletConfig)
  - pageContext